READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))


all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")



train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

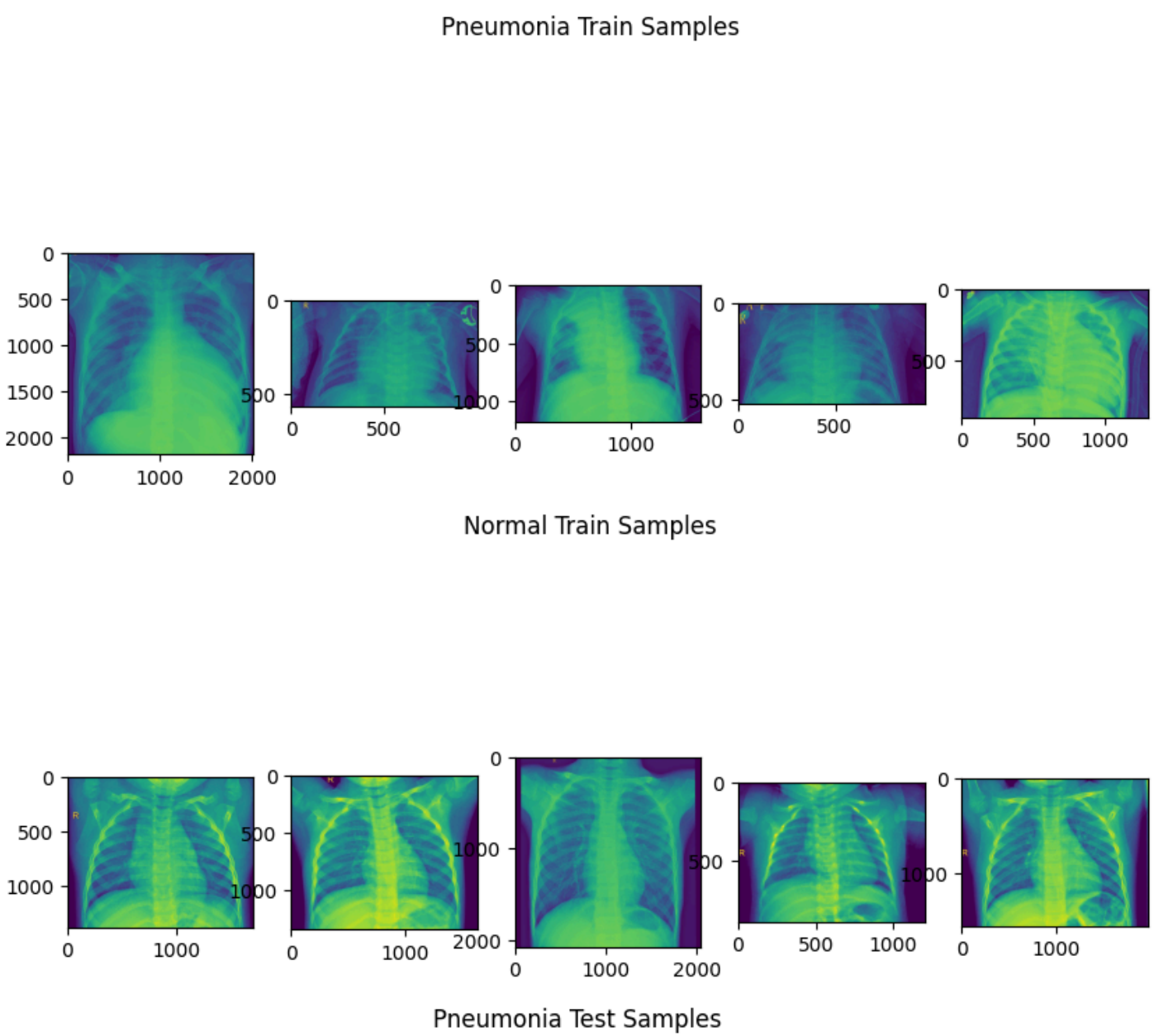
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

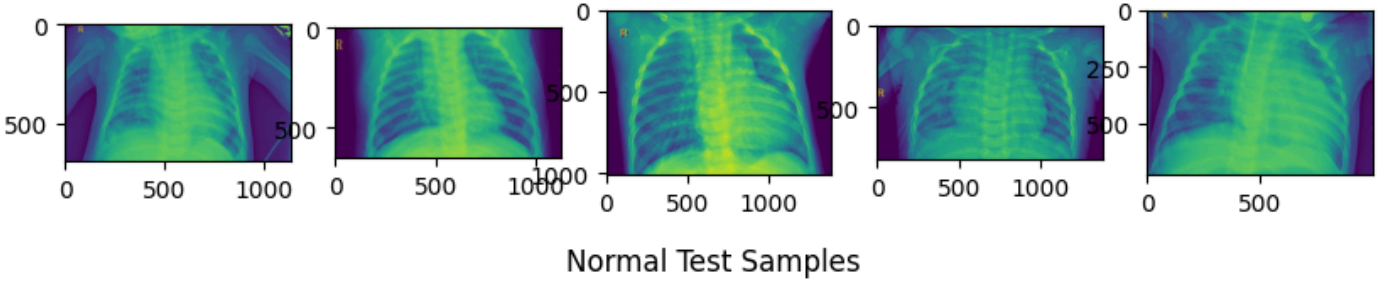
#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]

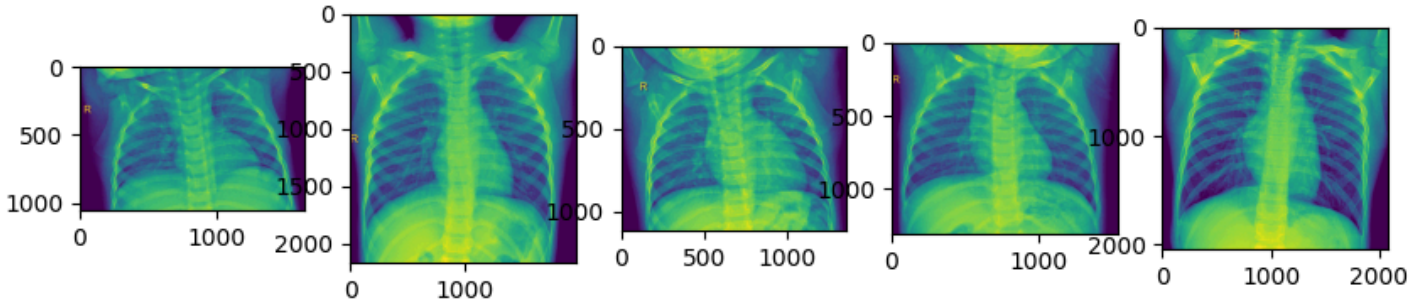

from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```


Pneumonia Train Samples
Normal Train Samples
Pneumonia Test Samples


Normal Test Samples



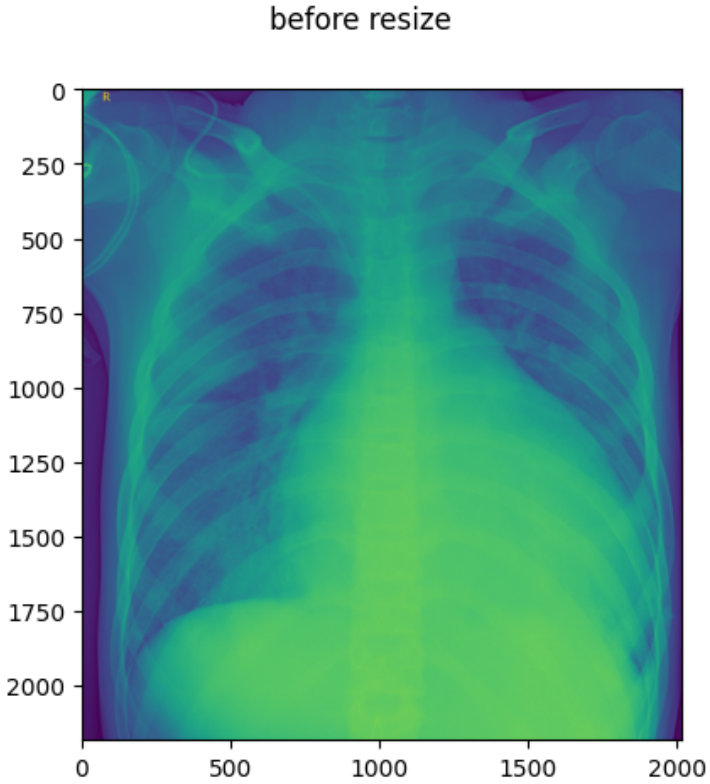
RESIZE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

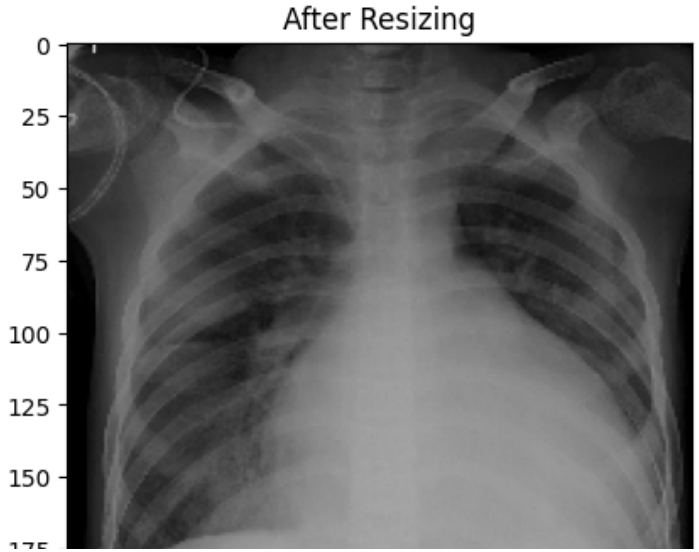
def load_images(file_paths, target_size=(216, 216)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

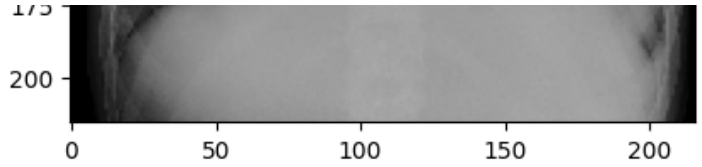
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)


show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```




```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.1176
Text(0.5, 1.0, 'After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```

```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(561.6577553453313),
 'contrast_std': np.float64(219.54756815948943),
 'contrast_var': np.float64(48201.13468474565),
 'dissimilarity_mean': np.float64(7.725462941868884),
 'dissimilarity_std': np.float64(2.093708148477257),
 'dissimilarity_var': np.float64(4.383613811000063),
 'homogeneity_mean': np.float64(0.23894586949885402),
 'homogeneity_std': np.float64(0.039918434776174946),
 'homogeneity_var': np.float64(0.0015934814349797333),
 'energy_mean': np.float64(0.02457923518556547),
 'energy_std': np.float64(0.0016259309769987476),
 'energy_var': np.float64(2.643651541964102e-06),
 'correlation_mean': np.float64(0.8884018691024708),
 'correlation_std': np.float64(0.045308143750558756),
 'correlation_var': np.float64(0.0020528278901212962),
 'ASM_mean': np.float64(0.0006067824538493038),
 'ASM_std': np.float64(7.998164329600663e-05),
 'ASM_var': np.float64(6.397063264329642e-09),
 'entropy': np.float64(141.46412676601597)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'
```

```
#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)
```

```
# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=35, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.3,
    callbacks=None,
```

```
        class_weight=class_weights,
    )
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,560
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 1)	65

```
 Total params: 11,649 (45.50 KB)
 Trainable params: 11,265 (44.00 KB)
 Non-trainable params: 384 (1.50 KB)
Epoch 1/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - accuracy: 0.6285 - auc: 0.7179 - loss: 0.7409 - val_accuracy: 0.7687 - val_auc: 0.0000e+00 - val_loss: 0.6256
Epoch 2/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8295 - auc: 0.9045 - loss: 0.4244 - val_accuracy: 0.7038 - val_auc: 0.0000e+00 - val_loss: 0.6583
Epoch 3/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8485 - auc: 0.8987 - loss: 0.4280 - val_accuracy: 0.8068 - val_auc: 0.0000e+00 - val_loss: 0.5314
Epoch 4/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8693 - auc: 0.9398 - loss: 0.3299 - val_accuracy: 0.7729 - val_auc: 0.0000e+00 - val_loss: 0.5827
Epoch 5/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8477 - auc: 0.9130 - loss: 0.3881 - val_accuracy: 0.8646 - val_auc: 0.0000e+00 - val_loss: 0.4218
Epoch 6/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8767 - auc: 0.9328 - loss: 0.3418 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.4020
Epoch 7/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8764 - auc: 0.9439 - loss: 0.3136 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3660
Epoch 8/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8769 - auc: 0.9372 - loss: 0.3235 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3719
Epoch 9/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8929 - auc: 0.9533 - loss: 0.2850 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3625
Epoch 10/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8791 - auc: 0.9458 - loss: 0.3008 - val_accuracy: 0.9027 - val_auc: 0.0000e+00 - val_loss: 0.3291
Epoch 11/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8765 - auc: 0.9375 - loss: 0.3281 - val_accuracy: 0.8984 - val_auc: 0.0000e+00 - val_loss: 0.3332
Epoch 12/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8754 - auc: 0.9378 - loss: 0.3309 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3688
```

```
Epoch 13/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9075 - auc: 0.9582 - loss: 0.2663 - val_accuracy: 0.8717 - val_auc: 0.0000e+00 - val_loss: 0.3713
Epoch 14/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8974 - auc: 0.9507 - loss: 0.2891 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3571
Epoch 15/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9015 - auc: 0.9536 - loss: 0.2744 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3484
Epoch 16/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8810 - auc: 0.9579 - loss: 0.2746 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3465
Epoch 17/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8896 - auc: 0.9456 - loss: 0.2990 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3408
Epoch 18/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8961 - auc: 0.9528 - loss: 0.2782 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3257
Epoch 19/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9032 - auc: 0.9547 - loss: 0.2790 - val_accuracy: 0.8702 - val_auc: 0.0000e+00 - val_loss: 0.3707
Epoch 20/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8896 - auc: 0.9493 - loss: 0.2890 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3183
Epoch 21/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9097 - auc: 0.9597 - loss: 0.2526 - val_accuracy: 0.9027 - val_auc: 0.0000e+00 - val_loss: 0.3258
Epoch 22/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8924 - auc: 0.9451 - loss: 0.3066 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3682
Epoch 23/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9017 - auc: 0.9557 - loss: 0.2725 - val_accuracy: 0.8688 - val_auc: 0.0000e+00 - val_loss: 0.3897
Epoch 24/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8961 - auc: 0.9536 - loss: 0.2773 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3385
Epoch 25/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9092 - auc: 0.9622 - loss: 0.2484 - val_accuracy: 0.8745 - val_auc: 0.0000e+00 - val_loss: 0.3782
Epoch 26/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9050 - auc: 0.9634 - loss: 0.2504 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3481
Epoch 27/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8896 - auc: 0.9561 - loss: 0.2702 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3588
Epoch 28/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9166 - auc: 0.9646 - loss: 0.2469 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3227
Epoch 29/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9138 - auc: 0.9596 - loss: 0.2605 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3439
Epoch 30/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9171 - auc: 0.9653 - loss: 0.2384 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3527
Epoch 31/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9032 - auc: 0.9620 - loss: 0.2525 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3564
Epoch 32/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8879 - auc: 0.9430 - loss: 0.3120 - val_accuracy: 0.8984 - val_auc: 0.0000e+00 - val_loss: 0.3311
Epoch 33/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9123 - auc: 0.9611 - loss: 0.2513 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3387
Epoch 34/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8925 - auc: 0.9553 - loss: 0.2705 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3351
Epoch 35/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9126 - auc: 0.9636 - loss: 0.2487 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3638
Epoch 36/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8956 - auc: 0.9601 - loss: 0.2583 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3218
Epoch 37/50
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8923 - auc: 0.9553 - loss: 0.2724 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3157
Epoch 38/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8990 - auc: 0.9584 - loss: 0.2619 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3401
Epoch 39/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8997 - auc: 0.9625 - loss: 0.2504 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3486
Epoch 40/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8885 - auc: 0.9562 - loss: 0.2719 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3372
Epoch 41/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9108 - auc: 0.9656 - loss: 0.2383 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3769
Epoch 42/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9039 - auc: 0.9596 - loss: 0.2606 - val_accuracy: 0.9013 - val_auc: 0.0000e+00 - val_loss: 0.3273
Epoch 43/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8965 - auc: 0.9547 - loss: 0.2684 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3501
Epoch 44/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8973 - auc: 0.9534 - loss: 0.2725 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3621
Epoch 45/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9178 - auc: 0.9729 - loss: 0.2202 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3398
Epoch 46/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9097 - auc: 0.9625 - loss: 0.2500 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3642
Epoch 47/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9057 - auc: 0.9594 - loss: 0.2540 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3604
Epoch 48/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9007 - auc: 0.9654 - loss: 0.2429 - val_accuracy: 0.9013 - val_auc: 0.0000e+00 - val_loss: 0.3328
Epoch 49/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9081 - auc: 0.9618 - loss: 0.2559 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3536
Epoch 50/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9072 - auc: 0.9636 - loss: 0.2441 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3523
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
- Loss: 0.2817
- AUC: 0.9594
- Accuracy: 0.8883
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.90      0.88      0.89       394
           1       0.88      0.90      0.89       394

    accuracy                           0.89       788
   macro avg       0.89      0.89      0.89       788
weighted avg       0.89      0.89      0.89       788
```

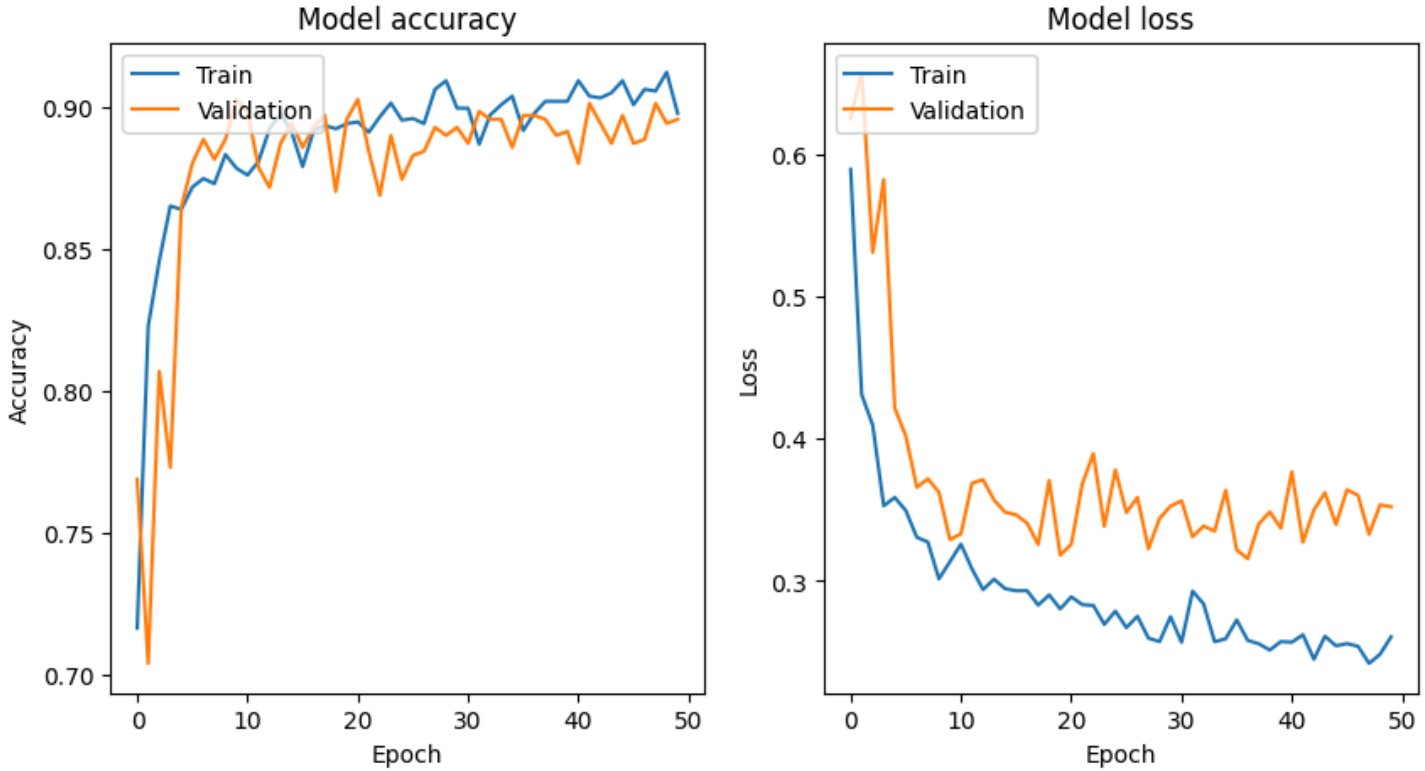
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

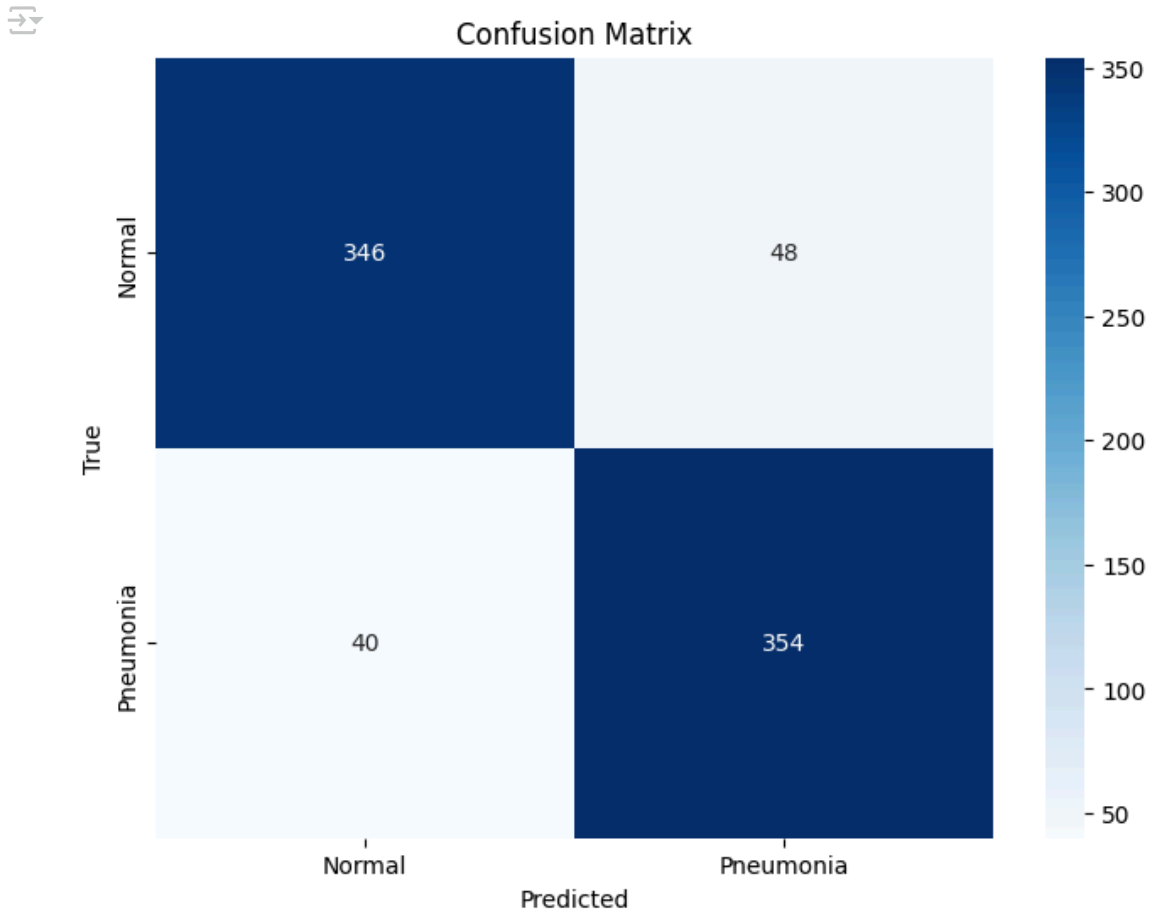
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
```

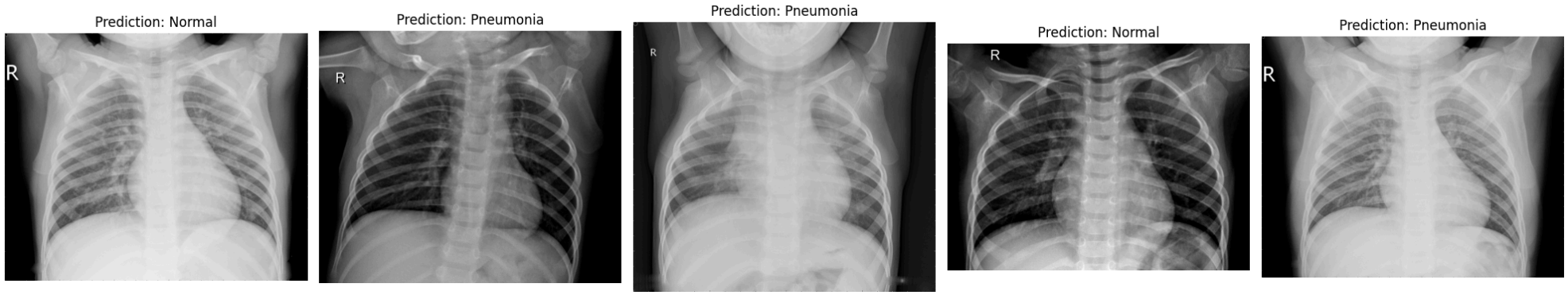
```
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Pneumonia
2	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Pneumonia
3	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Pneumonia
4	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Pneumonia
5	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
7	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal




```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
```

```
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

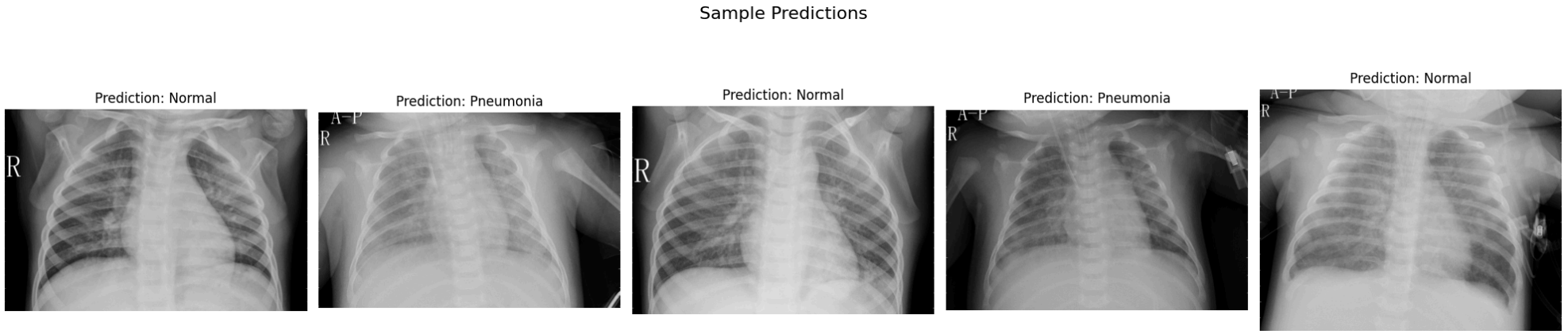
    except Exception as e:
```

```
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia
1	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia
7	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia




```
# prompt: uji data glcm ini juga dengan svm

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

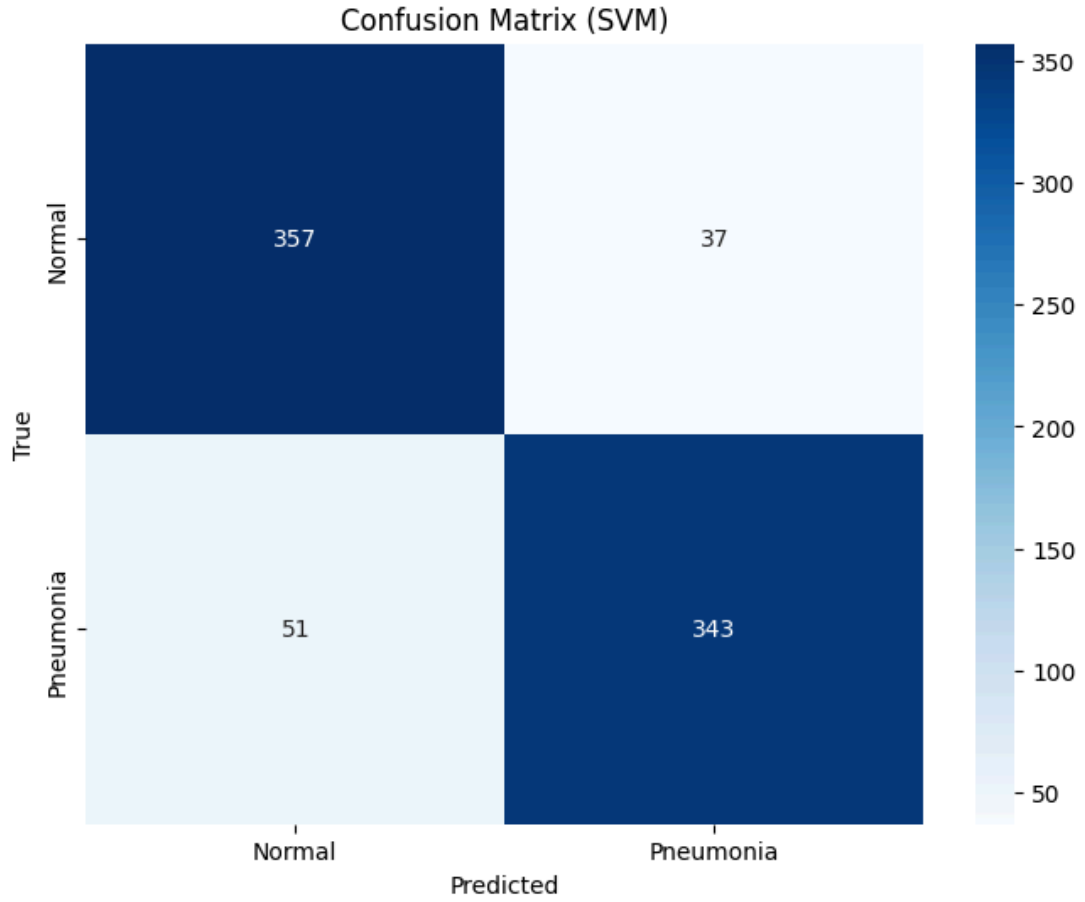
# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8883

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.91      0.89       394
           1       0.90      0.87      0.89       394

    accuracy                           0.89       788
   macro avg       0.89      0.89      0.89       788
weighted avg       0.89      0.89      0.89       788
```



```
# prompt: uji juga dengan random forest

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Create and train the Random Forest model
# You can adjust n_estimators (number of trees) and other parameters
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the Random Forest model
print("\n--- Random Forest Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# Confusion Matrix
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (Random Forest)')
plt.show()
```